

Angular Moderno

28 - Login com JWT

28 — Login com JWT (simulado)

Nesta aula vamos implementar uma estrutura de autenticação profissional no Angular moderno, cobrindo todos os pilares essenciais de segurança e estado reativo.

AuthService

Controle centralizado de autenticação

Signals

Estado global reativo

Guard

Proteção de rotas

Interceptor

Token integrado ao HTTP

Tela de Login

Interface funcional de autenticação

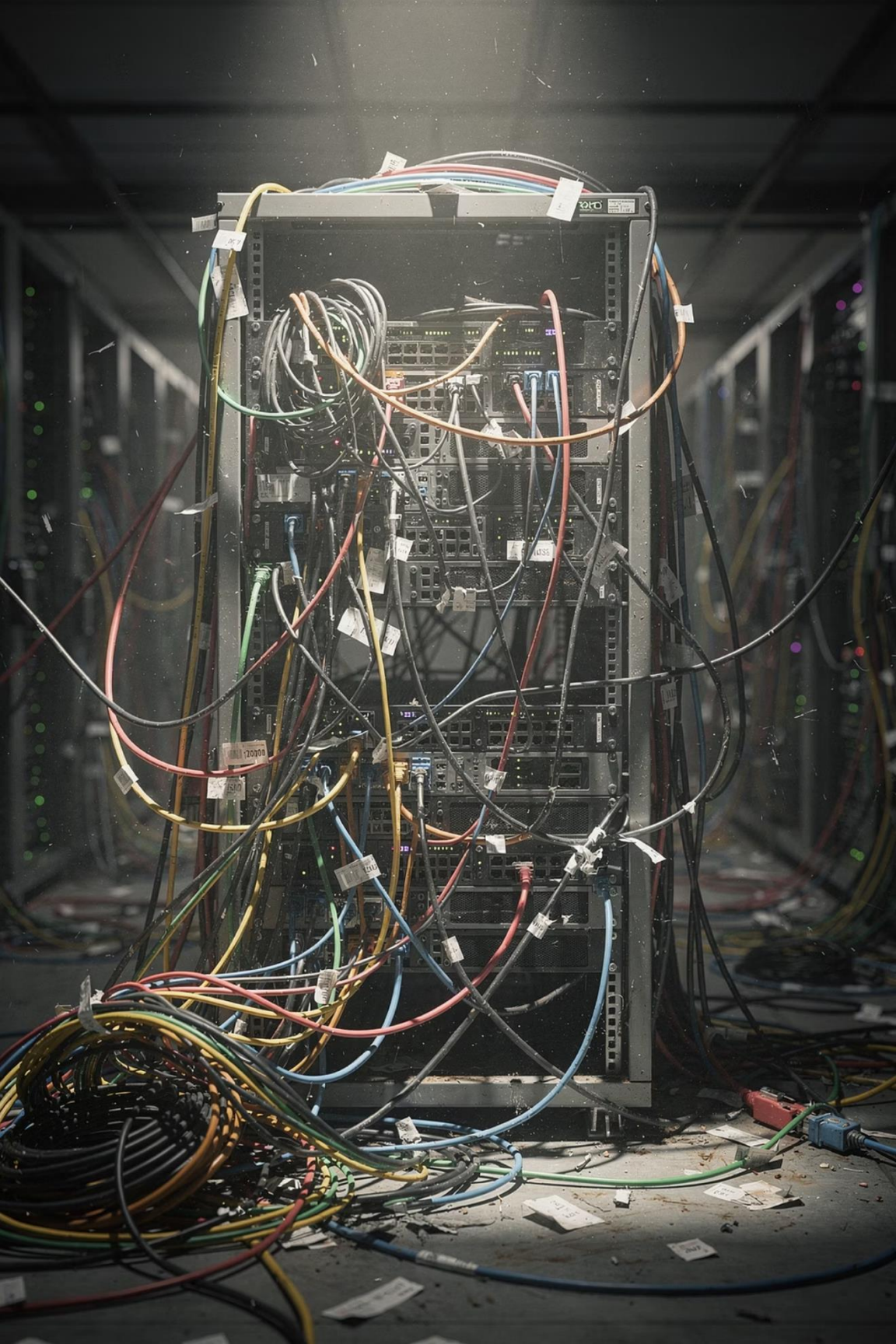
Contexto atual do projeto

→ Carrinho com estado global (Signals + Service) já implementado

→ Checkout funcional com formulário e validações

→ Interceptor HTTP configurado e operacional

→ Autenticação atual simplificada — será evoluída para estrutura profissional



Problema Atual da Aplicação

Situação Atual

- Autenticação baseada em lógica simples (arquivo `auth.ts`)
- Estado de login não estruturado
- Sem separação de responsabilidades
- Interceptor não integrado com autenticação real
- Guard dependente de função global

Objetivo da Evolução

- Criar uma arquitetura profissional de autenticação
- Centralizar estado no `AuthService`
- Integrar autenticação com Guard e Interceptor

 A refatoração transforma uma solução improvisada em uma arquitetura escalável e profissional.

Etapa 0 – Parar o Servidor

⚠ Ação obrigatória antes de qualquer alteração nos arquivos do projeto!

Comando

CTRL + C

Execute no terminal antes de prosseguir

Por que parar o servidor?

- 1 Evita erros
Previne conflitos durante alterações em múltiplos arquivos simultaneamente
- 2 Reconhecimento correto
Garante que o Angular reconheça corretamente todas as mudanças realizadas
- 3 Previne inconsistências
Evita problemas durante a refatoração estrutural do projeto

Etapa 1 — Remover Autenticação Antiga

Arquivo a excluir

`src/app/core/auth.ts`

⊗ Este arquivo deve ser completamente removido do projeto.
Não será mais utilizado a partir desta aula.

Justificativa

- Implementação simplificada sem padrão arquitetural
- Não segue o padrão de arquitetura com Services
- Será substituído pelo `AuthService` profissional

Importante

A partir desta aula:

- **NÃO** utilizamos mais `auth.ts`
- Toda autenticação será centralizada no service

📄 Esta é uma mudança de paradigma: saímos de funções soltas para um Service centralizado e reativo.

Etapa 2 – Criar AuthService

Novo arquivo

```
src/app/core/services/auth.service.ts
```



Centralizar Estado

Gerencia o estado de autenticação de forma única e confiável em toda a aplicação



Simular JWT

Simula a criação de um token após o login.



Signals Reativos

Expõe estado reativo com Signals para atualização automática da UI



Token para Interceptor

Fornece o token de autenticação para uso no interceptor HTTP

Código Completo — `auth.service.ts`

SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS

```
import { Injectable, computed, signal } from '@angular/core';

type Usuario = {
  email: string;
  perfil: 'usuario';
};

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  private usuario = signal<Usuario | null>(null);
  private tokenJwt = signal<string | null>(null);

  usuarioAtual = computed(() => this.usuario());
  estaLogado = computed(() => this.usuario() !== null);
  token = computed(() => this.tokenJwt());

  login(email: string, senha: string): boolean {
    if (!email || !senha) {
      return false;
    }
  }
}
```

```
const tokenSimulado =
  'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.' +
  'eyJzdWIiOiJhbHVub0B0ZXN0ZS5jb20iLCJwZXJmaWwiOiJ1c3VhcmlvIn0.' +
  'assinatura-simulada';

this.usuario.set({
  email,
  perfil: 'usuario',
});

this.tokenJwt.set(tokenSimulado);

return true;
}

logout() {
  this.usuario.set(null);
  this.tokenJwt.set(null);
}

obterToken(): string | null {
  return this.tokenJwt();
}
}
```

Etapa 3 — Atualizar Guard

SRC/APP/CORE/AUTH.GUARD.TS

ANTES

```
import { usuarioLogado } from './auth';
```

⚠ Dependência de função global: frágil e não escalável

Justificativa da mudança

Remove dependência global

Elimina o acoplamento com função global `usuarioLogado()`

Fonte única de verdade

Passa a usar `AuthService` como única fonte de estado de autenticação

Redirecionamento inteligente

Permite redirecionamento automático para a tela de login quando não autenticado

⚠ Veja o código completo de `auth.guard.ts` a seguir

DEPOIS

```
const authService = inject(AuthService);
const router = inject(Router);

if (authService.estaLogado()) {
  return true;
}

return router.createUrlTree(['/login']);
```

Código Completo — `auth.guard.ts`

SRC/APP/CORE/AUTH.GUARD.TS

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaLogado()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

✔ O Guard agora é uma função pura que delega toda a lógica de autenticação ao AuthService, seguindo o princípio de responsabilidade única.

Etapa 4 — Criar Tela de Login

Comando Angular CLI

```
ng generate component features/login/login
```

Estrutura criada

```
src/app/features/login/login/  
├─ login.ts  
├─ login.html  
└─ login.css
```

Objetivo

Criar a interface de autenticação do usuário com formulário reativo, validações e integração com o AuthService.

Etapa 5: Código Completo — login.ts

SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.TS

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';

import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);

  erroLogin = signal(false);

  formulario = new FormGroup({
    email: new FormControl('', [Validators.required, Validators.email]),
    senha: new FormControl('', [Validators.required, Validators.minLength(3)]),
  });
```

```
  entrar() {
    this.erroLogin.set(false);

    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      return;
    }

    const email = this.formulario.value.email ?? '';
    const senha = this.formulario.value.senha ?? '';

    const loginRealizado = this.authService.login(email, senha);

    if (!loginRealizado) {
      this.erroLogin.set(true);
      return;
    }

    this.router.navigateByUrl('/produtos');
  }
}
```

Etapa 6: Código Completo — login.html

SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.HTML

```
<h2>Login</h2>

<p>Informe seus dados para simular a autenticação na loja.</p>

@if (erroLogin()) {
  <p style="color: red">Não foi possível realizar o login.</p>
}

<form [formGroup]="formulario">
  <p>
    <input type="email" formControlName="email" placeholder="E-mail" />
  </p>

  @if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
    @if (formulario.get('email')?.errors?.['required']) {
      <small>E-mail é obrigatório</small>
    }

    @if (formulario.get('email')?.errors?.['email']) {
      <small>E-mail inválido</small>
    }
  }

  <p>
    <input type="password" formControlName="senha" placeholder="Senha" />
  </p>

  @if (formulario.get('senha')?.touched && formulario.get('senha')?.invalid) {
    @if (formulario.get('senha')?.errors?.['required']) {
      <small>Senha é obrigatória</small>
    }

    @if (formulario.get('senha')?.errors?.['minlength']) {
      <small>Senha deve ter no mínimo 3 caracteres</small>
    }
  }

  <p>
    <button type="button" (click)="entrar()" [disabled]="formulario.invalid">Entrar</button>
  </p>
</form>
```

Etapa 7 — Atualizar Rotas

SRC/APP/APP.ROUTES.TS

Nova rota de login

```
{
  path: 'login',
  loadComponent: () => import('./features/login/login/login').then((m) => m.Login),
},
```

Rotas protegidas com Guard

```
// Checkout protegido
{
  path: 'checkout',
  canActivate: [authGuard],
  loadComponent: ...
}

// Carrinho protegido
{
  path: 'carrinho',
  canActivate: [authGuard],
  loadComponent: ...
}
```

O que muda?

- Rota `/login` adicionada como ponto de entrada de autenticação
- Rotas `/checkout` e `/carrinho` agora exigem autenticação via `canActivate`
- Lazy loading mantido para performance otimizada

Código Completo — `app.routes.ts`

SRC/APP/APP.ROUTES.TS

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/auth.guard';

export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('../features/home/home/home').then((m) => m.Home),
  },
  {
    path: 'produtos',
    loadComponent: () => import('../features/produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
  {
    path: 'carrinho',
    canActivate: [authGuard],
    loadComponent: () => import('../features/carrinho/carrinho/carrinho').then((m) => m.Carrinho),
  },
  {
    path: 'checkout',
    canActivate: [authGuard],
    loadComponent: () => import('../features/checkout/checkout/checkout').then((m) => m.Checkout),
  },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login/login').then((m) => m.Login),
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```

Etapa 8 — Atualizar Interceptor

SRC/APP/CORE/INTERCEPTORS/HTTP.INTERCEPTOR.TS

ANTES — Token fixo

```
const token = 'fake-jwt-token';

const novaReq = req.clone({
  headers: {
    Authorization: `Bearer ${token}`,
  },
});
```

⚠ Token estático não reflete o estado real de autenticação do usuário

DEPOIS — Token do AuthService

```
const authService = inject(AuthService);
const token = authService.obterToken();

const novaReq = token
  ? req.clone({
      headers: {
        Authorization: `Bearer ${token}`,
      },
    })
  : req;
```

✅ O interceptor agora é dinâmico: injeta o token apenas quando o usuário está autenticado, e passa a requisição sem token caso contrário.

⚠ Veja o código completo de `http.interceptor.ts` a seguir.

Código Completo — http.interceptor.ts

SRC/APP/CORE/INTERCEPTORS/HTTP.INTERCEPTOR.TS

```
import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { tap, catchError, throwError } from 'rxjs';

import { AuthService } from '../services/auth.service';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.obterToken();

  // LOG REQUEST
  console.log('REQUEST', req.url);

  // TOKEN
  const novaReq = token
    ? req.clone({
        headers: {
          Authorization: `Bearer ${token}`,
        },
      })
    : req;
```

```
// SEGUE COM A NOVA REQUEST + LOG RESPONSE
return next(novaReq).pipe(
  tap({
    next: (event) => console.log('RESPONSE:', event),
    error: (error) => console.error('ERRO:', error),
  }),
  catchError((error) => {
    console.error('ERRO GLOBAL:', error);

    if (error.status === 401) {
      console.warn('Não autorizado!');
    }

    if (error.status === 500) {
      console.warn('Erro interno do servidor!');
    }

    return throwError(() => error);
  })
);
```

 O interceptor funcional (`HttpInterceptorFn`) é a abordagem moderna do Angular standalone, sem necessidade de classe ou decorador.

Etapa 9 — Atualizar Header

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.TS

Alterações necessárias

1 Injetar AuthService

Adicionar

`inject(AuthService)` no componente para acesso ao estado de autenticação

2 Expor estado no template

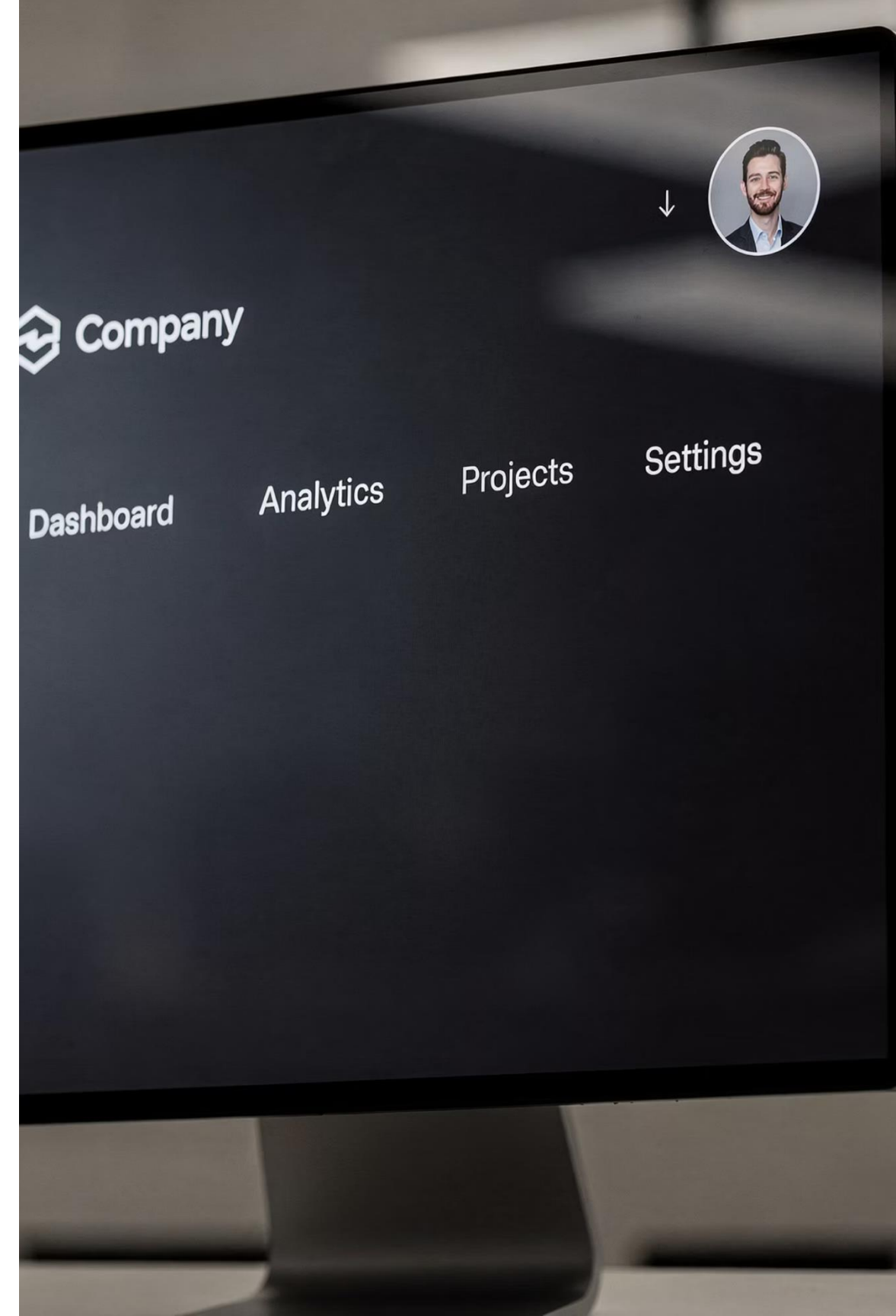
Disponibilizar `usuarioAtual()` e `estaLogado()` para uso no HTML do header

Resultado esperado

- Header exibe o e-mail do usuário quando logado
- Botão sair visível apenas para usuários autenticados
- Atualização automática via Signals sem necessidade de detecção manual de mudanças

- 📄 Os Signals garantem que o header se atualize automaticamente sempre que o estado de autenticação mudar.

⚠️ Veja o código completo de `header.ts` a seguir.



Código Completo — header.ts

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.TS

```
import { Component, inject } from '@angular/core';
import { RouterLink } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';

import { CarrinhoService } from '../../../core/services/carrinho.service';
import { AuthService } from '../../../core/services/auth.service';

@Component({
  selector: 'app-header',
  imports: [MatToolbarModule, MatButtonModule, RouterLink],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {
  private carrinhoService = inject(CarrinhoService);
  private authService = inject(AuthService);

  quantidade = this.carrinhoService.quantidade;
  estaLogado = this.authService.estaLogado;
  usuarioAtual = this.authService.usuarioAtual;

  sair() {
    this.authService.logout();
  }
}
```

Etapa 10: Validação Logado. Código Completo — header.html

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.HTML

```
<mat-toolbar color="primary">
  <span>Minha Loja</span>
  <span class="flex-spacer"></span>
  <button mat-button routerLink="/produtos">Produtos</button>
  <button mat-button routerLink="/carrinho">Carrinho ({{ quantidade() }})</button>
  <button mat-button routerLink="/checkout">Checkout</button>

  @if (estaLogado()) {
    <span>{{ usuarioAtual()?.email }}</span>
    <button mat-button type="button" (click)="sair()">Sair</button>
  } @else {
    <button mat-button routerLink="/login">Entrar</button>
  }
</mat-toolbar>
```

Etapa 11 – Limpar App

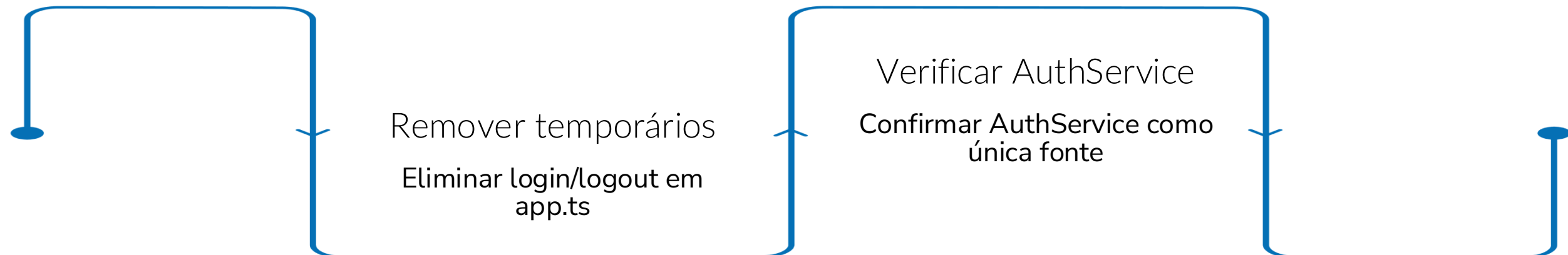
Esta etapa remove código temporário que foi substituído pela arquitetura profissional de autenticação.

Arquivo: `app.ts`

Remover os seguintes elementos:

- Atributo `login()` temporário
- Atributo `logout()` temporário
- Atributo `usuarioLogado`

Esses elementos foram migrados para o `AuthService`



⚠️ Veja o código completo de `app.ts` a seguir.

Código Completo — `app.ts`

SRC/APP/APP.TS

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

import { Header } from '../shared/layout/header/header';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {}
```

✅ O componente raiz agora está limpo, sem lógica de autenticação. Toda a responsabilidade foi delegada ao `AuthService` e ao `Header`.

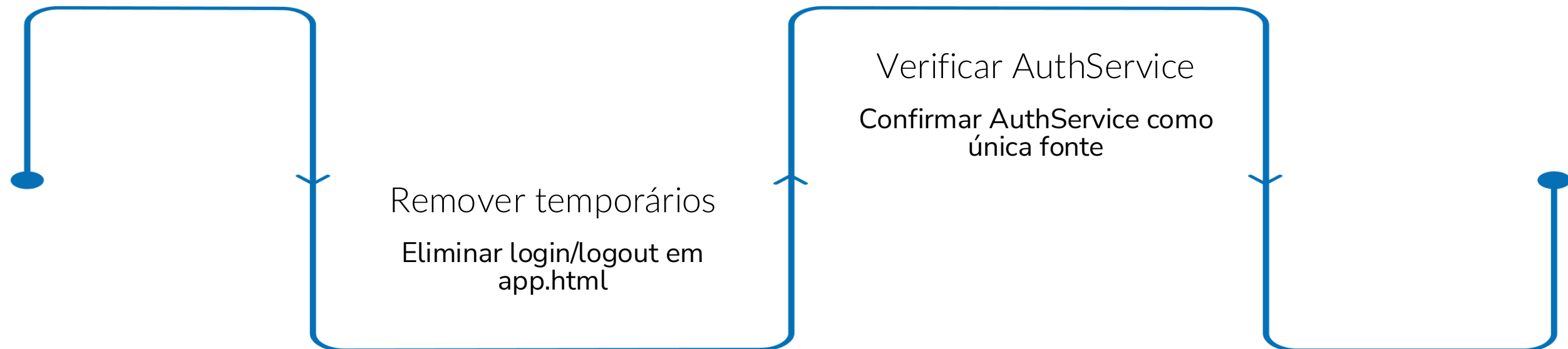
Etapa 12 – Limpar Template App

Arquivo: `app.html`

Remover os seguintes elementos:

- Botões de login temporários
- Qualquer lógica de autenticação inline no template

O `app.html` deve conter apenas o `router-outlet` e o componente de header



Veja o código completo de `app.ts` a seguir.

Código Completo — `app.html`

SRC/APP/APP.HTML

```
<app-header></app-header>

<main class="container-principal">
  <router-outlet></router-outlet>
</main>
```

 Template minimalista e limpo. O `app.html` agora contém apenas o header e o outlet de rotas — sem nenhuma lógica de autenticação inline.

Antes da limpeza

- Botões de login/logout temporários
- Bindings de estado de autenticação
- Lógica misturada no template raiz

Depois da limpeza

- Apenas `<app-header></app-header>`
- Apenas `<router-outlet></router-outlet>`
- Arquitetura limpa e profissional



Etapa 13 — Executar o Projeto

Comando para iniciar

```
ng serve
```

Acessar no navegador

```
http://localhost:4200/login
```

O que esperar

- Servidor Angular iniciado sem erros de compilação
- Tela de login exibida ao acessar `/login`
- Redirecionamento automático ao tentar acessar rotas protegidas
- Header reativo exibindo estado de autenticação

📄 Credenciais de teste sugeridas:
usuario@email.com / 123456

Testes Obrigatórios

Realize todos os testes abaixo para validar a implementação completa da autenticação:

1

Rota /carrinho sem login

Acessar `/carrinho` sem estar autenticado → deve redirecionar para `/login`

2

Rota /checkout sem login

Acessar `/checkout` sem estar autenticado → deve redirecionar para `/login`

3

Login válido

Inserir credenciais corretas → deve redirecionar para `/produtos`

4

Header reativo

Após login, o header deve exibir o email do usuário autenticado.

5

Logout funcional

Clicar em "Sair" limpa o estado de autenticação e o header volta a exibir o botão "Entrar".

6

Interceptor ativo

Após login, verificar no DevTools/console que as requisições continuam funcionando e que o interceptor usa o token quando ele existe.

Resultado da Aula

Ao concluir esta aula, a aplicação conta com uma **estrutura inicial de autenticação profissional**:



AuthService

Implementado com Signals e JWT simulado.



Login Funcional

Com formulário reativo.



Guard Integrado

Protegendo carrinho e checkout.



Interceptor Integrado

Ao token simulado.



Header Reativo

Exibindo o estado de autenticação.

✅ A arquitetura de autenticação está pronta! O projeto agora segue padrões profissionais com separação de responsabilidades, estado reativo e segurança de rotas.

Dúvidas

?

